



KANDIDAT

10096

PRØVE

INFT2503 1 C++ for programmerere

Emnekode	INFT2503
Vurderingsform	Hjemmeeksamen
Starttid	24.11.2025 08:00
Sluttid	24.11.2025 12:00
Sensurfrist	15.12.2025 23:59
PDF opprettet	24.11.2025 18:33

Seksjon 1

Oppgave	Tittel	Oppgavetype
i	Forside	Informasjon eller ressurser
1	Oppg. 1 (10%)	Programmering
2	Oppg. 2 (20%)	Programmering
3	Oppg. 3 (20%)	Programmering
4	Oppg. 4 (30%)	Programmering
5	Oppg. 5 (20%)	Programmering

1 Oppg. 1 (10%)

Bruk av generativ kunstig intelligens (f.eks. språkmodeller) **anses som juks** i denne oppgaven.

Lag nødvendig funksjon(er) slik at følgende kildekode:

```
cout << times_two(2) << endl;
cout << times_two(2.25) << endl;
cout << times_two("test") << endl;
cout << times_two(vector<int>{1, 2, 3}) << endl;
```

Gir utskriften:

```
4
4.5
testtest
{ 2, 4, 6 }
```

Skriv svaret ditt her. Endringer blir lagret automatisk.

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <type_traits>
5
6
7  // Aritmetiske typer
8  template <typename T>
9  constexpr typename std::enable_if<std::is_arithmetic<T>::value, T>::type
10 times_two(T x) {
11     return x * 2;
12 }
13
14
15 // Streng
16 std::string times_two(std::string s) {           // PBV, ettersom at typen ikke er
17     return s + s;                               // veldig stor i dette tilfellet
18 }
19
20 // Vector
21 std::vector<int> times_two(const std::vector<int>& v) {
22     std::vector<int> r;
23
24     r.reserve(v.size());
25
26     for (int x : v) {
27         r.emplace_back(x * 2);
28     }
29
30     return r;
31 }
32
33
34 // Utskrift av vector; - operator<< overload
35 template <typename T>
36 std::ostream& operator<<(std::ostream& os, const std::vector<T>& v) {
37     os << "{ ";
38
39     for (std::size_t i = 0; i < v.size(); ++i) {
40         os << v[i];
41
42         if (i + 1 < v.size()) {
43             os << ", ";
44         }
45     }
46 }
```

```
45     }
46
47     os << " }";
48
49     return os;
50 }
51
52
53 // Usikker på om jeg bare skulle implementere funksjonene, eller om jeg
54 // skulle inkludere main. Inkluderer derfor main for sikkerhetens skyld.
55 int main() {
56     std::cout << times_two(2) << std::endl;           // Jeg hadde nok heller valgt
57     std::cout << times_two(2.25) << std::endl;         // for å unngå å flushe, men
58     std::cout << times_two("test") << std::endl;       // men macher oppgaven for si
59     std::cout << times_two(std::vector<int>{1, 2, 3}) << std::endl;
60
61     return 0;
62 }
63
64
65
66
67
68
```

Maks poeng: 10

2 Oppg. 2 (20%)

Bruk av generativ kunstig intelligens (f.eks. språkmodeller) **anses som juks** i denne oppgaven.

a) Lag nødvendige klasse(r) slik at følgende kildekode:

```
vector<unique_ptr<ProgrammingLanguage>> programming_languages;
// Uncommenting the following line should cause compilation error:
// programming_languages.emplace_back(make_unique<ProgrammingLanguage>
//   ());
programming_languages.emplace_back(make_unique<Cpp>());
programming_languages.emplace_back(make_unique<Rust>());

for (auto &programming_language : programming_languages) {
    cout << programming_language->print_line("Hello World") << endl;
    cout << programming_language->for_each("e", "vec") << endl
        << endl;
}
```

Gir utskriften:

```
cout << "Hello World" << endl;
for(auto &e: vec) {}
```

```
println!("Hello World");
for e in &vec {}
```

b) Utvid løsningen du skrev i oppgave **a)** slik at kildekoden:

```
vector<unique_ptr<ProgrammingLanguage>> programming_languages;
programming_languages.emplace_back(make_unique<Cpp>(make_unique<CMake>
  ()));
programming_languages.emplace_back(make_unique<Cpp>(make_unique<Meson>
  ()));
programming_languages.emplace_back(make_unique<Rust>
  (make_unique<Cargo>()));

for (auto &programming_language : programming_languages) {
    cout << programming_language->build() << endl;
}
```

Gir utskriften:

```
cmake --build .
meson compile -C .
cargo build
```

Skriv svaret ditt her. Endringer blir lagret automatisk.

1	#include <iostream>
2	#include <memory>
3	#include <vector>

```

4  #include <string>
5  #include <utility>
6
7
8  // Abstrakt base klasse
9  class BuildSystem {                                // Strengt tatt, kunne d
10 public:                                             // structs, ettersom alt
11     virtual ~BuildSystem() {}
12     virtual std::string buildCommand() const = 0;
13 };
14
15
16 class CMake : public BuildSystem {
17 public:
18     std::string buildCommand() const override { return "cmake --build ."; }
19 };
20
21 class Meson : public BuildSystem {
22 public:
23     std::string buildCommand() const override { return "meson compile -C ."; }
24 };
25
26 class Cargo : public BuildSystem {
27 public:
28     std::string buildCommand() const override { return "cargo build"; }
29 };
30
31
32
33 // Abstrakt base klasse for programmeringsspråk
34 class ProgrammingLanguage {
35 protected:
36     std::unique_ptr<BuildSystem> buildSystem_;
37
38 public:
39     ProgrammingLanguage() {}
40     explicit ProgrammingLanguage(std::unique_ptr<BuildSystem> bs) : buildSystem_
41     virtual ~ProgrammingLanguage() {}
42
43     // Disse holder ProgrammingLanguage abstrakt slik at forsøk på å instansiere
44     virtual std::string print_line(const std::string& s) const = 0;
45     virtual std::string for_each(const std::string& elementName, const std::stri
46
47     // Default implementasjon som delegerer til et valgfritt BuildSystem
48     virtual std::string build() const {
49         if (buildSystem_) {
50             return buildSystem_>buildCommand();
51         }
52
53         return std::string();
54     }
55 };
56
57
58 // C++ implementasjon
59 class Cpp : public ProgrammingLanguage {
60 public:
61     Cpp() : ProgrammingLanguage() {}
62     explicit Cpp(std::unique_ptr<BuildSystem> bs) : ProgrammingLanguage(std::mov
63
64     std::string print_line(const std::string& s) const override {
65         return "cout << \"" + s + "\" << endl;";
66     }
67
68     std::string for_each(const std::string& elementName, const std::string& cont
69         return "for(auto &" + elementName + ": " + containerName + ") {}";
70     }
71 };
72
73
74 // Rust implementasjon
75 class Rust : public ProgrammingLanguage {
76 public:
77     Rust() : ProgrammingLanguage() {}

```

```

77     Rust() : ProgrammingLanguage() {}
78     explicit Rust(std::unique_ptr<BuildSystem> bs) : ProgrammingLanguage(std::move(bs)) {}
79
80     std::string print_line(const std::string& s) const override {
81         return "println!\"" + s + "\"";
82     }
83
84     std::string for_each(const std::string& elementName, const std::string& containerName) const override {
85         return "for " + elementName + " in " + containerName + " {}";
86     }
87 };
88
89
90 int main() {
91     // NB: Jeg har valgt å implementere begge delene i hver sin scope
92     //       for å unngå at variablene fra a) skal forstyrre i b)
93
94     // a)
95     {
96         std::vector<std::unique_ptr<ProgrammingLanguage>> programming_languages;
97         // Uncommenting the following line should cause compilation error:
98         // programming_languages.emplace_back(std::make_unique<ProgrammingLanguage>());
99
100        programming_languages.emplace_back(std::make_unique<Cpp>());
101        programming_languages.emplace_back(std::make_unique<Rust>());
102
103        for (auto &programming_language : programming_languages) {
104            std::cout << programming_language->print_line("Hello World") << std::endl;
105            std::cout << programming_language->for_each("e", "vec") << std::endl;
106        }
107    }
108
109    // b)
110    {
111        std::vector<std::unique_ptr<ProgrammingLanguage>> programming_languages;
112        programming_languages.emplace_back(std::make_unique<Cpp>());
113        programming_languages.emplace_back(std::make_unique<Cpp>());
114        programming_languages.emplace_back(std::make_unique<Rust>());
115
116        for (auto &programming_language : programming_languages) {
117            std::cout << programming_language->build() << std::endl;
118        }
119    }
120
121    return 0;
122 }
123

```

Maks poeng: 20

3 Oppg. 3 (20%)

Bruk av generativ kunstig intelligens (f.eks. språkmodeller) **anses som juks** i denne oppgaven.

Lag klassen `Angle` slik at følgende kildekode:

```
cout << Angle<int>(30) << endl;
cout << (Angle<int>(30) + 360) << endl;
cout << (Angle<int>(30) + 3.14159265359) << endl;
cout << Angle<double>(3.14159265359 / 2) << endl;
cout << (Angle<double>(3.14159265359 / 2) + 3.14159265359) << endl;
cout << (Angle<double>(3.14159265359 / 2) + 3) << endl;
```

Gir utskriften:

```
30
30
33
1.5708
1.5708
1.4292
```

Skriv svaret ditt her. Endringer blir lagret automatisk.

```
1  #include <iostream>
2  #include <type_traits>
3  #include <cmath>
4
5
6  template <typename T>
7  class Angle {
8      static_assert(std::is_arithmetic<T>::value, "Angle krever aritmetisk type");
9
10 private:
11     T value_;
12
13     // Normaliseringsfunksjon: grader hvis integral-type, altså tall uten desimal
14     static T normalize(T v) {
15         if (std::is_integral<T>::value) {
16             // Normaliser til [0, 360)
17             long long vv = static_cast<long long>(v);
18             long long m = vv % 360;
19
20             if (m < 0) {
21                 m += 360;
22             };
23
24             return static_cast<T>(m);
25
26         } else {
27             // Normaliser til [0, 2*pi)
28             double dv = static_cast<double>(v);
29             double mod = std::fmod(dv, 2.0 * M_PI);
30
31             if (mod < 0) {
32                 mod += 2.0 * M_PI;
33             };
34
35             return static_cast<T>(mod);
36         }
37     }
38 }
```

```
39 public:
40     // Konstruer fra T
41     explicit Angle(T v) : value_(normalize(v)) {}
42
43     // Konstruer fra annen aritmetisk type; - konverteres til T
44     template <typename U, typename = typename std::enable_if<std::is_arithmetic<U
45     Angle(U v) : value_(normalize(static_cast<T>(v))) {}
46
47     T value() const { return value_; }
48
49     // Legg til en aritmetisk verdi; resultatet er fortsatt Angle<T>
50     template <typename U, typename = typename std::enable_if<std::is_arithmetic<U
51     Angle operator+(U rightHandSide) const {
52         T sum = value_ + static_cast<T>(rightHandSide);
53
54         return Angle(sum); // konstruktøren normaliserer igjen
55     }
56
57 };
58
59
60 // Utskriftsoperator: viser den normaliserte verdien med standard stream-format
61 template <typename T>
62 std::ostream& operator<<(std::ostream& os, Angle<T> const& a) {
63     os << a.value();
64
65     return os;
66 }
67
68
69 int main() {
70     std::cout << Angle<int>(30) << std::endl;
71     std::cout << (Angle<int>(30) + 360) << std::endl;
72     std::cout << (Angle<int>(30) + 3.14159265359) << std::endl;
73     std::cout << Angle<double>(3.14159265359 / 2) << std::endl;
74     std::cout << (Angle<double>(3.14159265359 / 2) + 3.14159265359) << std::endl;
75     std::cout << (Angle<double>(3.14159265359 / 2) + 3) << std::endl;
76
77     return 0;
78 }
```

Maks poeng: 20

4 Oppg. 4 (30%)

Bruk av generativ kunstig intelligens (f.eks. språkmodeller) **anses som juks** i denne oppgaven.

I denne oppgaven skal du lage kode som utfører filtrering av input i flere tråder parallelt. Hver tråd skal lagre resultat sitt i en egen `vector`. Alle `vector`'ene skal til slutt returneres.

Du kan bruke `thread::hardware_concurrency()` for å finne antall CPU kjerner som er tilgjengelig.

Lag nødvendige klasser og funksjoner slik at følgende kildekode:

```
vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
cout << (vec | ParallellFilter([](int e) { return e % 2 == 0; })) <<
endl;
cout << (vec | ParallellFilter([](int e) { return e % 2 == 1; })) <<
endl;
```

Gir for eksempel utskriften:

```
{ { }, { 2, 10 }, { }, { 4 }, { }, { 6 }, { }, { 8 } }
{ { 1, 9 }, { }, { 3 }, { }, { 5 }, { }, { 7 }, { } }
```

Skriv svaret ditt her. Endringer blir lagret automatisk.

```
1 #include <iostream>
2 #include <vector>
3 #include <thread>
4 #include <functional>
5 #include <type_traits>
6
7
8 // Fabrikk-wrapper-type som holder predikatet
9 template <typename Pred>
10 class ParallellFilterImpl { // Denne klas
11 public:
12     Pred pred;
13     explicit ParallellFilterImpl(Pred p) : pred(std::move(p)) {}
14 };
15
16
17 // Deduction hjelper-funksjon slik at kalleren kan skrive ParallellFilter(lambda)
18 template <typename Pred>
19 ParallellFilterImpl<typename std::decay<Pred>::type> ParallellFilter(Pred&& p) {
20     return ParallellFilterImpl<typename std::decay<Pred>::type>(std::forward<Pred>
21 }
22
23
24 // operator|: anvender ParallellFilter på en container. Merk; forventer random-ac
25 template <typename Container, typename Pred>
26 auto operator|(const Container& c, const ParallellFilterImpl<Pred>& pf)
27     -> std::vector<std::vector<typename Container::value_type>> // Her har je
28     det
29 { // enklere å
30     lang
31     unsigned int hw = std::thread::hardware_concurrency();
32     std::size_t nthreads = (hw == 0 ? 1u : static_cast<std::size_t>(hw)); /
33
34     std::vector<std::vector<typename Container::value_type>> results(nthreads);
35
36     // Start opp tråder: hver tråd prosesserer indekser i = tid, tid + nthreads,
37     std::vector<std::thread> threads;
38     threads.reserve(nthreads);
```

```

36     threads.receive(nthreads);
37
38     for (std::size_t tid = 0; tid < nthreads; ++tid) {
39         threads.emplace_back([&, tid]() {
40             for (std::size_t i = tid; i < c.size(); i += nthreads) {
41                 if (pf.pred(c[i])) {
42                     results[tid].emplace_back(c[i]); // hvor hver tråd skriver til
43                     // synkronisering nødvendig
44                 }
45             }
46         });
47
48     for (auto& t : threads) {
49         if (t.joinable()) t.join();
50     }
51
52     return results;
53 }
54
55
56 // Utskriftsoperator: viser innholdet i en std::vector<T> på form: { elem1, elem2
57 // Fungerer rekursivt for nestede vektorer.
58 template <typename T>
59 std::ostream& operator<<(std::ostream& os, const std::vector<T>& v) {
60     os << "{ ";
61
62     for (std::size_t i = 0; i < v.size(); ++i) {
63         os << v[i];
64
65         if (i + 1 < v.size()) {
66             os << ", ";
67         }
68     }
69
70     os << " }";
71
72     return os;
73 }
74
75
76 int main() {
77     std::vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
78
79     // Note: nummeret av indre vektorer er lik std::thread::hardware_concurrency(
80     std::cout << (vec | ParallellFilter([](int e) { return e % 2 == 0; })) << std
81     std::cout << (vec | ParallellFilter([](int e) { return e % 2 == 1; })) << std
82
83     return 0;
84 }
85

```

Maks poeng: 30

5 Oppg. 5 (20%)

I denne oppgaven **kan du bruke** generativ kunstig intelligens (f.eks. språkmodeller). Hvis du bruker generativ kunstig intelligens, må du **henvise** til KI-verktøyet du brukte.

Bruk gtkmm til å lage en enkel kalkulator.

Krav:

- koden må kompilere
- et vindu med en enkel kalkulator må vises når applikasjonen kjøres
- følgende operasjoner må fungere i kalkulatoren: +, -, *, /

Skriv svaret ditt her. Endringer blir lagret automatisk.

```
1 // KI-hjelp: Rammestruktur generert av GitHub Coplot (ChatGPT GPT-5 mini) 2025-1
  implementasjon rettet opp i av GitHub Coplot (ChatGPT GPT-5 mini) 2025-11-24
2 // TODO: FIKSE DESIMALTALL MULTIPLIKASJONS AVRUNDINGSFEIL
3 // TODO: LEGGE TIL PARENTESER I UI
4
5 #include <gtkmm.h>
6 #include <iostream>
7 #include <sstream>
8 #include <iomanip>
9 #include <cmath>
10 #include <string>
11 #include <vector>
12 #include <stack>
13 #include <cctype>
14 #include <stdexcept>
15
16
17 class CalculatorWindow : public Gtk::Window {
18 private:
19     // UI
20     Gtk::Box vbox_{Gtk::ORIENTATION_VERTICAL};
21     Gtk::Entry display_;
22     Gtk::Grid grid_;
23
24     // State
25     char selectedOperator = '%';
26     operator
27     bool unaryMinusUsed = true;
28
29     // * Helpers
30 void addButton(const std::string& label, int col, int row, int width = 1, in
31     auto btn = Gtk::manage(new Gtk::Button(label));
32     btn->signal_clicked().connect(sigc::bind<std::string>(sigc::mem_fun(*thi
33     ), label));
34
35     grid_.attach(*btn, col, row, width, height);
36
37 void addText(const std::string& text) {
38     std::string current = display_.get_text();
39     display_.set_text(current + text);
40 }
41
42 void removeText(int count) {
43     std::string current = display_.get_text();
44
45     // Safety check, in case of invalid count
46 if (current.size() < static_cast<size_t>(count)) {
47     display_.set_text("");
```

```

48         return;
49     }
50
51     std::string newText = current.substr(0, current.size() - count);
52     display_.set_text(newText);
53 }
54
55 void changeText(const std::string& text, int backtrackIndicies) {
56     removeText(backtrackIndicies);
57     addText(text);
58 }
59
60 const char lastTextCharacter() const {
61     std::string current = display_.get_text();
62
63     // Safety check
64     if (current.empty()) {
65         std::cout << "Display is empty.\n";
66         return '\0';
67     }
68
69     return current.back();
70 }
71
72
73 const bool validUnaryCase(const char operand) const {
74     if (operand != '-') return false;
75
76     return (
77         display_.get_text().empty() ||
78         isOperand(lastTextCharacter())
79     );
80 }
81
82 // * Static functions
83 static bool isOperand(char c) {
84     return c == '+' || c == '-' || c == '*' || c == '/';
85 }
86
87 static int prec(const std::string &op) {
88     if (op == "u") return 4;
89     if (op == "*" || op == "/" ) return 3;
90     if (op == "+" || op == "-") return 2;
91     return 0;
92 };
93
94 void onButtonClicked(const std::string& label) {
95     switch (static_cast<char>(label[0])) {
96         case 'C':
97             clear();
98             break;
99
100         case '+': case '-': case '*': case '/':
101             std::cout << "Operator button clicked: " << label << "\n";
102
103             selectedOperator = label[0];
104
105             if (lastTextCharacter() == '-' && selectedOperator == '-') {
106                 removeText(1);
107                 unaryMinusUsed = false;
108
109             } else if (validUnaryCase(selectedOperator)) {
110                 // unary minus case
111                 addText(label);
112                 unaryMinusUsed = true;
113
114             } else if (display_.get_text().empty()) {
115                 return;
116
117             } else if (isOperand(lastTextCharacter())) {
118                 changeText(label, 1 + unaryMinusUsed);
119                 unary minus if used
120                 unaryMinusUsed = false;

```

```

121     } else {
122         // append operator
123         addText(label);
124     }
125
126     break;
127
128     // ! NO BUTTON YET
129 case '(':
130     std::cout << "Left parenthesis button clicked\n";
131
132     if (!display_.get_text().empty() &&
133         (std::isdigit(static_cast<unsigned char>(lastTextCharacter())
134         // implicit multiplication
135         addText("*(");
136     } else {
137         addText("(");
138     }
139
140     unaryMinusUsed = true;
141
142     break;
143
144     // ! NO BUTTON YET
145 case ')':
146     std::cout << "Right parenthesis button clicked\n";
147
148     if (!display_.get_text().empty() &&
149         (std::isdigit(static_cast<unsigned char>(lastTextCharacter())
150         addText(")");
151     }
152
153     unaryMinusUsed = false;
154
155     break;
156
157 case '=':
158     std::cout << "Equals button clicked" << "\n";
159
160     unaryMinusUsed = false;
161     // Handle equals button click here
162     evaluateExpression();
163
164     break;
165
166 case '.':
167     std::cout << "Decimal point button clicked" << "\n";
168     unaryMinusUsed = false;
169     // Handle decimal point button click here
170     break;
171
172
173 default:
174     // Ensure label is a digit
175     if (!(label.size() == 1 && std::isdigit(static_cast<unsigned cha
176         throw std::runtime_error("Invalid number: " + label);
177         return;
178     }
179
180     std::cout << label << " digit clicked" << "\n";
181
182     // Add operator and digit to display string
183     // digit: append, replacing a single leading "0"
184     addText(label);
185     unaryMinusUsed = false;
186
187     break;
188 }
189
190
191 std::cout << "Current display: " << display_.get_text() << "\n";
192 }
193

```

```

194
195 void evaluateExpression() {
196     auto s = display_.get_text();
197     if (s.empty()) return;
198
199     std::vector<std::string> tokens;
200     ( ) ; unary minus -> "u"
201
202     for (size_t i = 0; i < s.size(); ) {
203         char c = s[i];
204         // if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue;
205         // allow spaces
206
207         // Number
208         if (std::isdigit(static_cast<unsigned char>(c)) || c == '.') {
209             size_t j = i;
210
211             while (j < s.size() && (std::isdigit(static_cast<unsigned char>(
212                 tokens.emplace_back(s.substr(i, j - i));
213
214             i = j;
215
216             continue;
217         }
218
219         // Operator or parenthesis
220         if (c == '+' || c == '*' || c == '/' || c == '-' || c == '(' || c ==
221             if (c == '-') {
222                 // unary if at start or after operator or '('
223                 bool unary = tokens.empty() ||
224                     tokens.back() == "+" || tokens.back() == "-" ||
225                     tokens.back() == "*" || tokens.back() == "/" ||
226                     tokens.back() == "(";
227                 if (unary) { tokens.emplace_back("u"); ++i; continue; }
228             }
229             tokens.emplace_back(1, c);
230             ++i;
231             continue;
232         }
233
234         // invalid char
235         display_.set_text("Error"); return;
236
237         // Shunting-yard -> RPN
238
239         auto isRightAssociative = [](const std::string &op)->bool { return op ==
240         auto isUorOperand = [](const std::string &t)->bool { return t == "+" ||
241             == "u"; };
242
243         std::vector<std::string> output;
244         std::vector<std::string> ops;
245
246         for (const auto &tok : tokens) {
247             if (!tok.empty() && (std::isdigit(static_cast<unsigned char>(tok[0])
248                 output.emplace_back(tok);
249
250             } else if (isUorOperand(tok)) {
251                 while (!ops.empty() && isUorOperand(ops.back())) {
252                     std::string top = ops.back();
253
254                     if ((!isRightAssociative(tok) && prec(tok) <= prec(top)) ||
255                         < prec(top)) {
256                         output.emplace_back(top); ops.pop_back();
257                     } else break;
258                 }
259                 ops.emplace_back(tok);
260             } else if (tok == "(") {
261                 ops.emplace_back(tok);
262             } else if (tok == ")") {

```

```
264         bool found = false;
265
266         while (!ops.empty()) {
267             if (ops.back() == "(") {
268                 ops.pop_back();
269                 found = true;
270                 break;
271             }
272
273             output.emplace_back(ops.back());
274             ops.pop_back();
275         }
276
277         if (!found) {
278             display_.set_text("Error");
279             return;
280         }
281
282     } else {
283         display_.set_text("Error");
284         return;
285     }
286 }
287
288 while (!ops.empty()) {
289     if (ops.back() == "(" || ops.back() == ")") {
290         display_.set_text("Error");
291         return;
292     }
293
294     output.emplace_back(ops.back()); ops.pop_back();
295 }
296
297 // Evaluate RPN
298 std::vector<double> st;
299
300 for (const auto &tok : output) {
301     if (tok == "u") {
302         if (st.empty()) {
303             display_.set_text("Error");
304             return;
305         }
306
307         double v = st.back();
308         st.pop_back();
309         st.emplace_back(-v);
310
311     } else if (tok == "+" || tok == "-" || tok == "*" || tok == "/") {
312         if (st.size() < 2) {
313             display_.set_text("Error");
314             return;
315         }
316
317         double b = st.back(); st.pop_back();
318         double a = st.back(); st.pop_back();
319
320         if (tok == "+") st.emplace_back(a + b);
321         else if (tok == "-") st.emplace_back(a - b);
322         else if (tok == "*") st.emplace_back(a * b);
323         else {
324             if (b == 0.0) { display_.set_text("Error"); return; }
325
326             st.emplace_back(a / b);
327         }
328
329     } else {
330         try {
331             st.emplace_back(std::stod(tok));
332         }
333
334         catch (...) {
335             display_.set_text("Error");
336             return;
337         }
338     }
```

```

338         }
339     }
340
341     if (st.size() != 1) { display_.set_text("Error"); return; }
342
343     // Format result (no trailing zeros)
344     double result = st.back();
345
346     if (std::isnan(result) || std::isinf(result)) { display_.set_text("Error"); return; }
347
348     std::ostringstream oss;
349     oss << std::setprecision(12) << std::fixed << result;
350     std::string out = oss.str();
351
352     if (out.find('.') != std::string::npos) {
353         while (!out.empty() && out.back() == '0') out.pop_back();
354         if (!out.empty() && out.back() == '.') out.pop_back();
355     }
356
357     if (out.empty()) out = "0";
358
359     display_.set_text(out);
360     unaryMinusUsed = false;
361 }
362
363 void clear() {
364     display_.set_text("");
365     unaryMinusUsed = false;
366     selectedOperator = '%';
367 }
368
369 public:
370 CalculatorWindow() {
371     set_title("GTKmm Calculator");
372     set_default_size(320, 240);
373
374     // Top-level layout
375     vbox_.set_spacing(6);
376     add(vbox_);
377
378     // Display
379     display_.set_editable(false);
380     display_.set_hexand(true);
381     display_.set_margin_start(6);
382     display_.set_margin_end(6);
383     display_.set_margin_top(6);
384     display_.set_margin_bottom(6);
385     display_.set_text("");
386     vbox_.pack_start(display_, Gtk::PackOptions::PACK_SHRINK);
387
388     // Grid for buttons
389     grid_.set_row_spacing(4);
390     grid_.set_column_spacing(4);
391     grid_.set_margin_start(6);
392     grid_.set_margin_end(6);
393     grid_.set_margin_top(6);
394     grid_.set_margin_bottom(6);
395     vbox_.pack_start(grid_);
396
397     // Buttons layout (rows x cols)
398     // Row/col layout:
399     // 7 8 9 /
400     // 4 5 6 *
401     // 1 2 3 -
402     // 0 . = +
403
404     addButton("7", 0, 0);
405     addButton("8", 1, 0);
406     addButton("9", 2, 0);
407     addButton("/", 3, 0);
408
409     addButton("4", 0, 1);
410     addButton("5", 1, 1);

```

```
411         addButton("6", 2, 1);
412         addButton("*", 3, 1);
413
414         addButton("1", 0, 2);
415         addButton("2", 1, 2);
416         addButton("3", 2, 2);
417         addButton("-", 3, 2);
418
419         addButton("0", 0, 3);
420         addButton(".", 1, 3);
421         addButton "=", 2, 3);
422         addButton("+", 3, 3);
423
424         auto clear_btn = Gtk::manage(new Gtk::Button("C"));
425         clear_btn->signal_clicked().connect(sigc::mem_fun(*this, &CalculatorWind
426         grid_.attach(*clear_btn, 0, 4, 4, 1);
427
428         show_all_children();
429     }
430
431     virtual ~CalculatorWindow() = default;
432 };
433
434 int main(int argc, char** argv) {
435     auto app = Gtk::Application::create(argc, argv);
436
437     CalculatorWindow window;
438
439     return app->run(window);
440 }
```

Maks poeng: 20